

Bubble API Usage

This document describes how the BlockPro Flutter app communicates with the Bubble.io backend API (the v2 API, introduced in commit `f29beef`).

Overview

Item	Detail
Base URL	Configured via the <code>BUBBLE_API_BASE_URL</code> env var (<code>.env</code> file). Currently points to the Bubble workflow API: <code>.../api/1.1/wf/</code>
HTTP client	Dart <code>http</code> package
Auth method	Bearer token in the Authorization header
Config file	<code>lib/core/config/api_config.dart</code>
HTTP helpers	<code>lib/repositories/api_repository.dart</code> — provides <code>authenticatedGet</code> , <code>authenticatedGetRaw</code> , and <code>authenticatedPost</code>

All authenticated requests include these headers:

```
Authorization: Bearer <token>
Content-Type: application/json
```

Endpoints

1. `app_login` — User Authentication

Path	<code>app_login</code>
Method	POST
Auth required	No
Source file	<code>lib/repositories/auth_repository.dart</code>

When it's called: User taps the login button on the login screen.

Request body:

```
{
  "email": "<email>",
  "password": "<password>"
}
```

Response:

```
{
  "status": "success",
  "response": {
    "token": "<bearer-token>",
    "user_id": "<user-id>",
    "expires": 86400
  }
}
```

What happens next:

- The bearer token is stored in-memory and persisted to `SharedPreferences` .
- Token expiry is calculated from the `expires` field (seconds from now).
- On app restart, the stored token is restored and checked for expiration. If expired, the user is logged out automatically.

2. `app_fetchbuildings` — Download Buildings

Path	<code>app_fetchbuildings</code>
Method	GET
Auth required	Yes
Source file	<code>lib/repositories/sync_repository.dart</code>

When it's called: Phase 1 of the initial sync after login. Also called on retry.

Query params: None.

Response: JSON array of building objects.

```
[
  {
    "id": "1756484476534x490740256824968800",
    "name": "Building 1"
  }
]
```

Parsed fields:

JSON key	Model field	Notes
<code>id</code>	<code>Building.id</code>	
<code>name</code>	<code>Building.name</code>	Falls back to "Unnamed"

What happens next: Buildings are upserted into the local SQLite `buildings` table via `buildingsDao.upsertBuildings()` .

Response format notes: The parser (`parseBuildingsResponse`) handles three response shapes defensively:

1. A direct JSON array (expected)
2. A map with a `response` key containing the array
3. A JSON string that needs a second decode

3. `app_fetch_all_assets` — Download Assets for a Building

Path	<code>app_fetch_all_assets</code>
Method	GET
Auth required	Yes
Source file	<code>lib/repositories/sync_repository.dart</code>

When it's called: Phase 2 of initial sync — one call per building.

Query params:

Param	Value
<code>block_id</code>	The building ID

Response: JSON array of asset objects.

```
[
  {
    "assetId": "1771864899143x375085884294525250",
    "buildingId": "1756831363197x720244312004630400",
    "taskname": "Fire door inspection",
    "assetnickname": "896",
    "assetregisteritems": "",
    "tooltiptext": "",
    "tooltipurls": "",
    "lastcompleted": "2026-04-14T11:27:56.319Z",
    "duedate": "2026-04-14T11:27:56.319Z",
    "frequency": "7 Day(s)",
    "colour": "Red",
    "location": "",
    "floor": "",
    "yellowdate": "2026-04-14T11:27:56.319Z",
    "assetlastmodified": "2026-04-13T10:09:31.738Z",
    "checklistlastmodified": "2026-04-14T11:02:50.833Z"
  }
]
```

Parsed fields:

JSON key	Model field	Notes
<code>assetId</code>	<code>Asset.id</code>	
<code>buildingId</code>	<code>Asset.buildingId</code>	Links the asset to its building

taskname	Asset.taskName	
assetnickname	Asset.nickname	Null if empty
colour	Asset.colour	Enum: Red, Yellow, Green
lastcompleted / duedate / yellowdate	dates	Parsed as DateTime
frequency	Asset.frequency	e.g. "7 Day(s)"
checklistlastmodified	(sync key)	Drives the incremental checklist fetch in Phase 3

What happens next: Assets are upserted into the local SQLite `assets` table, linked to their building via `buildingId`. Each asset's `checklistlastmodified` is captured to decide which checklists are stale.

4. `app_fetch_checklist_single` — Download Checklist for an Asset

Path	<code>app_fetch_checklist_single</code>
Method	GET
Auth required	Yes
Source file	<code>lib/repositories/sync_repository.dart</code>

When it's called: Phase 3 of the initial sync — one call per asset, run in parallel with a max concurrency of 5.

Incremental: a checklist is only fetched when its `checklistlastmodified` differs from the value stored locally.

Query params:

Param	Value
<code>asset_id</code>	The asset ID

Response: JSON array containing a single checklist object with a chapters → questions hierarchy.

```
[
  {
    "parentassetid": "1771864899143x375085884294525250",
    "chapters": [
      {
        "chaptername": "Fire door",
        "chapterorder": 1,
        "questions": [
          {
            "questionid": "1771871963645x392498056957566000",
            "questiontext": "Does this door self-close fully into its frame?",
            "questiondesc": "Additional guidance",
            "answertype": "Yes|No",
            "photorequirement": "Always",
            "questionordernumber": 5,
            "existingremedials": []
          }
        ]
      }
    ]
  }
]
```

```

    ]
  }
]
}
]
```

Parsed fields:

JSON key	Model field	Notes
chaptername	Chapter.name	
chapterorder	Chapter.order	
questionid	Question.id	
questiontext	Question.questionText	
questiondesc	Question.description	
questionordernumber	Question.orderNumber	
answertype	Question.answerOption	Enum: Yes No, Yes No N/A, Satisfactory Unsatisfactory, Satisfactory Unsatisfactory N/A
photorequirement	Question.photoRequirement	Enum: Always, Only when unsatisfactory
existingremedials	Question.existingRemedials	Array; stored as a JSON blob. Remedial fields: remedialname, remedialdesc, remedialallocation, remedialduedate, remedialpriority (Low / High)

What happens next: Chapters are upserted via `chaptersDao.upsertChapters()` and questions (with their remedials) via `questionsDao.upsertQuestions()`.

5. app_upload-image — Upload Inspection Photo

Path	app_upload-image
Method	POST
Auth required	Yes
Source file	lib/providers/inspection_provider.dart

When it's called: During inspection submission — before the final inspection payload is sent. Each photo is uploaded individually.

Request body:

```

{
  "base64": "<base64-encoded-image-bytes>",
  "asset_id": "<asset-id>",
```

```
"filename": "photo_2026-03-01.jpg"
}
```

Response:

```
{
  "response": {
    "image_id": "img_abc123",
    "image_url": "<url>"
  }
}
```

What happens next:

- The returned `image_id` is collected into a list.
- If upload fails, the error is logged but submission continues (photos are best-effort).
- All collected `image_id` values are included in the `app_completed-inspection` payload.

6. `app_completed-inspection` — Submit Completed Inspection

Path	<code>app_completed-inspection</code>
Method	POST
Auth required	Yes
Source file	<code>lib/providers/inspection_provider.dart</code>

When it's called: When the user taps "Submit" on the inspection screen, after all photos have been uploaded.

Request body:

```
{
  "asset_id": "<asset-id>",
  "answers": [
    {
      "question_text": "Does this door self-close fully into its frame?",
      "answer_text": "Yes"
    },
    {
      "question_text": "Is the seal intact?",
      "answer_text": "No"
    }
  ],
  "photo_ids": ["img_abc123", "img_def456"]
}
```

- `photo_ids` is only included when at least one photo was successfully uploaded.
- `answer_text` is the selected label (e.g., `"Yes"` , `"No"` , `"Satisfactory"` , `"N/A"`), or an empty string if no answer was selected.

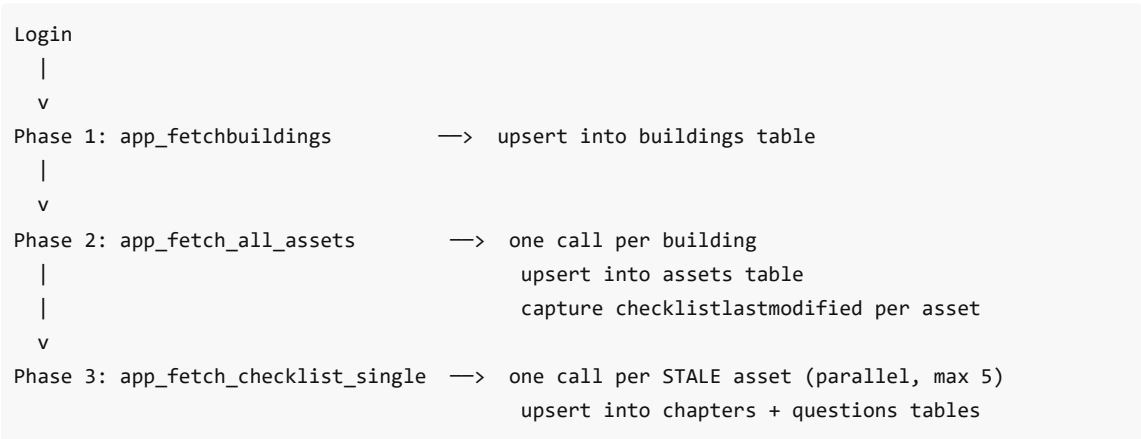
Validation (client-side, before submission):

- Every question with answer options must have a selected answer.
- If a photo is required (based on `photorequirement` rules and the selected answer), at least one photo must be attached.

Data Sync Flow

Initial Sync (after login)

Orchestrated by `SyncRepository.syncAll()` , driven by `InitialSyncNotifier` .



The initial sync screen (`lib/screens/initial_sync_screen.dart`) shows progress through each phase. A retry button is available if any phase fails.

Data Storage

All synced data is stored locally in a SQLite database (via Drift ORM). The app reads from the local database for all UI display, using the API only to populate/refresh the local cache.

Connectivity & Offline Handling

Source file: `lib/services/connectivity_service.dart`

- Combines hardware connectivity checks (`connectivity_plus` package) with API success/failure signals.
- `reportApiSuccess()` / `reportApiFailure()` are called after every API request by `ApiRepository` .
- Connectivity changes are debounced (1 second) to prevent UI flicker.
- The app tracks both network interface availability and API reachability.

Key Files

File	Purpose
<code>lib/core/config/api_config.dart</code>	Base URL configuration from <code>.env</code>
<code>lib/repositories/api_repository.dart</code>	HTTP client with authenticated GET/POST helpers
<code>lib/repositories/auth_repository.dart</code>	Login, token storage, session management

lib/repositories/sync_repository.dart	Orchestrates all data sync (buildings, assets, checklists)
lib/utils/api_parsers.dart	Response parsing & normalization for all data endpoints
lib/providers/inspection_provider.dart	Photo upload & inspection submission
lib/providers/initial_sync_provider.dart	Initial sync UI state machine
lib/models/building.dart	Building model & JSON parsing
lib/models/asset.dart	Asset model & JSON parsing
lib/models/question.dart	Question/Chapter/Remedial models, answer & photo-requirement enums